Master 2 TNAH, École nationale des chartes
27 octobre 2026

Marion Charpier



Introduction au langage de programmation Python

Séance 5 - Lire et enregistrer des données : JSON

Programme du cours

Séance	Date	Sujet
1	29 septembre 2026	Les bases de la programmation
2	6 octobre 2026	Créer ses propres outils : fonctions et modules
3	14 octobre 2026	Programmation orientée objet 1 : créer ses propres objets
4	20 octobre 2026	Programmation orientée objet 2 : héritage et décorateurs
5	27 octobre 2026	Lire et enregistrer des données : JSON
6	3 novembre 2026	Du tableur à Python : CSV
7	10 novembre 2026	Explorer les collections numérisées : IIIF

Plan de la séance 5

1) **Correction de l'interrogation**
2) **Les décorateurs**
   a) Rappel : donner une fonction à une autre fonction
   b) Construire un décorateur
   c) Appliquer un décorateur : @
   d) Décorer une fonction qui a des paramètres : *args et **kwargs
   e) Pourquoi utiliser des décorateurs ?
3) **Le format JSON**
   a) Pourquoi enregistrer ses données ?
   b) La syntaxe JSON
   c) De Python à JSON, et retour
4) **Lire et enregistrer des fichiers**
   a) Ouvrir un fichier : with open()
   b) La bibliothèque json
   c) Les chemins avec pathlib
5) **Parcourir un manifeste IIIF**



Rappel : donner une fonction à une autre fonction

Rappel de la séance 2

Pour trier le corpus par date, on a écrit :

```
sorted(corpus, key=lambda manuscrit: manuscrit["date"])
```

- On a donné une fonction à sorted(), **sans l'appeler**.
- C'est sorted() qui l'appelle, pour chaque manuscrit du corpus.

Avec ou sans parenthèses

- siecle(1210) : on **appelle** la fonction et on obtient son résultat.
- siecle : on désigne **la fonction elle-même**, comme une valeur.

Une fonction est une valeur

```
def siecle(date: int) -> str:
    ...

siecle(1210)
# 'XIIIe siècle'

siecle
# <function siecle at 0x...>

f = siecle      # pas de parenthèses : on ne l'exécute pas
f(1210)
# 'XIIIe siècle'
```

Pourquoi c'est utile pour la suite

Un décorateur reçoit une fonction **sans parenthèses**, puis l'appelle lui-même au bon moment.

Construire un décorateur

L'objectif

Afficher un message avant et après afficher_titre(), **sans toucher à son code**.

Une image : la reliure

On ne réécrit pas les feuillets d'un manuscrit pour le relier : on les place **à l'intérieur** d'une reliure. Un décorateur fait la même chose : il place la fonction d'origine à l'intérieur d'une nouvelle fonction, qui ajoute quelque chose avant et après.

Le décorateur en trois temps

1. annoncer **reçoit** une fonction, sans parenthèses.
2. Elle **fabrique** une nouvelle fonction, qui affiche un message, appelle la fonction reçue, puis affiche un second message.
3. Elle **renvoie** cette nouvelle fonction, sans l'exécuter.

```
def annoncer(fonction):          # 1. reçoit la fonction
    def nouvelle_fonction():     # 2. fabrique la reliure
        print("Avant l'exécution de la fonction")
        resultat = fonction()    #    appelle la fonction d'origine
        print("Après l'exécution de la fonction")
        return resultat
    return nouvelle_fonction     # 3. renvoie la reliure
```

À ce stade, **rien ne s'affiche** : on a seulement défini annoncer.

Appliquer un décorateur

À la main

```
def afficher_titre():
    print(titre)

afficher_titre = annoncer(afficher_titre)
```

Cette dernière ligne se lit de droite à gauche :

1. on donne la fonction afficher_titre à annoncer ;
2. annoncer renvoie nouvelle_fonction ;
3. on la range sous le nom afficher_titre.

Désormais, afficher_titre désigne la fonction **reliée**.

Avec @

```
@annoncer
def afficher_titre():
    print(titre)
```

@annoncer est une **abréviation** de afficher_titre = annoncer(afficher_titre).

Ce qui se passe à l'appel

```
afficher_titre()
# Avant l'exécution de la fonction
# III, 14. De la colombe.
# Après l'exécution de la fonction
```

1. On appelle afficher_titre, qui est maintenant nouvelle_fonction.
2. Elle affiche « Avant… ».
3. Elle appelle fonction(), c'est-à-dire l'ancienne afficher_titre, qui affiche le titre.
4. Elle affiche « Après… ».

Décorer une fonction qui a des paramètres



Le problème

```
@annoncer
def siecle(date: int) -> str:
    ...

siecle(1210)
# TypeError: annoncer.<locals>.nouvelle_fonction()
# takes 0 positional arguments but 1 was given
```

- siecle a été remplacée par nouvelle_fonction, qui n'accepte **aucun argument** : la reliure n'a pas prévu de place pour 1210.
- Il faut que nouvelle_fonction accepte **n'importe quels arguments** et les transmette à la fonction d'origine.

Rappel de la séance 2

- *args regroupe les arguments **positionnels** dans un **tuple**.
- **kwargs regroupe les arguments **nommés** dans un **dictionnaire**.

La solution

```
def annoncer(fonction):
    def nouvelle_fonction(*args, **kwargs):
        print("Avant l'exécution de la fonction")
        resultat = fonction(*args, **kwargs)
        print("Après l'exécution de la fonction")
        return resultat
    return nouvelle_fonction
```

- Dans le def, les étoiles **regroupent** tous les arguments reçus.
- À l'appel de fonction, elles les **déballent** pour les transmettre tels quels.
- return resultat renvoie le résultat de siecle() : sans cette ligne, print() afficherait None.
- Le décorateur fonctionne avec **toutes les fonctions** : afficher_titre() sans argument, siecle(1210) avec un argument.

Pourquoi utiliser des décorateurs ?



Séparer les tâches

- La fonction fait son travail : siecle() calcule un siècle.
- Le décorateur se charge d'une tâche annexe : annoncer l'appel, vérifier les arguments, gérer les erreurs.
- Chaque partie a un seul rôle : le code est plus lisible.

Ne pas modifier ce qui fonctionne

- La fonction reste telle qu'elle a été écrite et testée : on ajoute, on ne modifie pas.

Réutiliser le code

- Un même décorateur s'applique à plusieurs fonctions : @annoncer décore afficher_titre() comme siecle(), sans dupliquer le code.
- Pour changer le comportement ajouté, on ne modifie que le décorateur : toutes les fonctions décorées en profitent.

Pourquoi enregistrer ses données ?



Ce qu'on a fait jusqu'ici

- Depuis la séance 1, on décrit les manuscrits du corpus
  - des dictionnaires (séance 1),
  - des fonctions pour les traiter (séance 2),
  - des objets Manuscrit (séances 3 et 4).
- Mais à chaque séance, il a fallu **relancer une cellule qui recrée toutes les données** au début du notebook.

Pourquoi ?

Les variables n'existent que **pendant l'exécution** du programme. Si l'on ferme le notebook, corpus disparaît :

```
print(corpus)
# NameError: name 'corpus' is not defined
```

Une image : le catalogue

- Une **variable**, c'est une notice qu'on garde en tête : elle disparaît quand on referme le notebook.
- Un **fichier**, c'est une notice inscrite au catalogue : on la retrouve plus tard, et d'autres peuvent la consulter.

Deux formats ce semestre

- **JSON** (aujourd'hui) : des données **imbriquées**, comme une notice qui contient la liste de ses créatures.
- **CSV** (séance 6) : un **tableau**, avec une ligne par manuscrit.

Les institutions publient elles aussi leurs données en JSON : c'est le cas des manifestes IIIF.

La syntaxe JSON

Qu'est-ce que JSON ?

- ***JavaScript Object Notation*** : un format **texte**, léger et lisible.
- Il est utilisé par la plupart des services web pour échanger des données.
- Il ne dépend d'aucun langage : Python, JavaScript ou R savent le lire.

Les éléments d'un fichier JSON

- Des **objets** { }, composés de paires **clé-valeur**.
- Des **listes** [ ], composées de valeurs ordonnées.
- Les clés sont toujours des **chaînes de caractères**.
- Les valeurs peuvent être :
  - des chaînes de caractères : "MS 24"
  - des nombres : 1200
  - des booléens : true / false
  - une valeur vide : null
  - des listes ou des objets, imbriqués les uns dans les autres.

La notice de MS 24 en JSON

```
{
  "cote": "MS 24",
  "bibliotheque": "Aberdeen, University Library",
  "date": 1200,
  "numerise": true,
  "nb_folios": null,
  "creatures": ["dragon", "licorne", "lion", "panthère", "phénix"]
}
```

Le corpus entier : une liste de notices

```
[
  {"cote": "MS 24", "date": 1200, ...},
  {"cote": "MS. Ashmole 1511", "date": 1210, ...}
]
```

Attention : les chaînes s'écrivent **toujours entre guillemets doubles**.

De Python à JSON, et retour

Deux opérations

- **Sérialiser** : transformer un objet Python en texte JSON, pour l'enregistrer.
- **Désérialiser** : relire du texte JSON pour retrouver un objet Python.

Les correspondances

Python	JSON
dict	objet { }
list	liste [ ]
str	chaîne "..."
int, float	nombre
True / False	true / false
None	null

```
manuscrit = {"cote": "MS 24", "date": 1200, "numerise": True,
"nb_folios": None} # devient en JSON : # {"cote": "MS 24", "date":
1200, # "numerise": true, "nb_folios": null}
```

Ce qui ne revient pas à l'identique

Un tuple revient sous forme de **liste** :

{"datation": (1200, 1210)} → {'datation': [1200, 1210]}

Une clé numérique revient sous forme de **chaîne** :

{1200: "MS 24"} → {'1200': 'MS 24'}

Ce qui ne passe pas du tout

Un set ou un objet Manuscrit :

```
# TypeError: Object of type set is not JSON serializable
# TypeError: Object of type Manuscrit is not JSON serializable
```

Il faut d'abord le convertir en liste ou en dictionnaire.

Ouvrir un fichier : with open()

Ouvrir un fichier

with open(nom_du_fichier, mode, encoding="utf-8") as fichier:

- with ferme le fichier **automatiquement** à la fin du bloc, même en cas d'erreur.
- encoding="utf-8" garantit que les accents sont bien lus et écrits, quel que soit le système (Windows, macOS, Linux).

Les modes d'ouverture

Mode	Description
"r"	Lire (FileNotFoundError si le fichier n'existe pas)
"w"	Écrire (crée le fichier, ou **écrase** son contenu)
"a"	Ajouter à la fin (crée le fichier s'il n'existe pas)
"wb"	Écriture binaire (images : séance 7)

Lire et écrire

- fichier.write(texte) : écrit une chaîne de caractères.
- fichier.read() : lit tout le contenu, sous forme de chaîne.

Exemple : enregistrer le titre

```
with open("colombe.txt", "w", encoding="utf-8") as fichier:
    fichier.write(titre)

with open("colombe.txt", "r", encoding="utf-8") as fichier:
    contenu = fichier.read()

print(contenu)
# III, 14. De la colombe.
```

La bibliothèque json

json est un module **intégré** à Python : il suffit d'écrire import json.

Avec un fichier

json.dump(objet, fichier) : enregistre un objet Python dans un fichier JSON.

```
with open("corpus.json", "w", encoding="utf-8") as fichier:
    json.dump(corpus, fichier, ensure_ascii=False, indent=4)
```

json.load(fichier) : relit un fichier JSON et renvoie un objet Python.

```
with open("corpus.json", "r", encoding="utf-8") as fichier:
    corpus = json.load(fichier)

print(corpus[0]["cote"])
# MS 24
```

Avec une chaîne de caractères

Le **s** final signifie *string* (chaîne).

- json.dumps(objet) : transforme un objet Python en chaîne JSON.
- json.loads(chaine) : transforme une chaîne JSON en objet Python.

```
manuscrit = json.loads('{"cote": "MS 24", "date": 1200}')
# {'cote': 'MS 24', 'date': 1200}
```

Options utiles (pour dump et dumps)

- indent=4 : présente le fichier avec des retraits, comme en slide 12.
- ensure_ascii=False : conserve les accents.

```
print(json.dumps({"creatures": ["lion", "panthère"]}))
# {"creatures": ["lion", "panth\u00e8re"]}

print(json.dumps({"creatures": ["lion", "panthère"]},
ensure_ascii=False))
# {"creatures": ["lion", "panthère"]}
```

Les chemins avec pathlib

Qu'est-ce que pathlib ?

- Un module **standard** qui représente les chemins de fichiers comme des **objets**, et non comme de simples chaînes.
- Le même code fonctionne sous Windows, macOS et Linux.

Construire un chemin

```
from pathlib import Path

dossier = Path("donnees")
chemin = dossier / "corpus.json"

print(chemin)
# donnees/corpus.json   (donnees\corpus.json sous Windows)
```

L'opérateur / assemble les morceaux avec le bon séparateur.

Les propriétés d'un chemin

- chemin.name : corpus.json
- chemin.stem : corpus
- chemin.suffix : .json
- chemin.parent : donnees

Vérifier et créer

- chemin.exists() : renvoie True si le fichier existe.
- dossier.mkdir(exist_ok=True) : crée le dossier, sans erreur s'il existe déjà.

Lire et écrire

- chemin.write_text(texte, encoding="utf-8") : écrit du texte.
- chemin.read_text(encoding="utf-8") : lit le texte.

Avec json

```
chemin.write_text(json.dumps(corpus, ensure_ascii=False, indent=4), encoding="utf-8")

corpus = json.loads(chemin.read_text(encoding="utf-8"))
```

Un objet Path s'utilise aussi avec open() : with open(chemin, "r", encoding="utf-8") as fichier:

Parcourir un manifeste IIIF

Un manifeste est un fichier JSON

Une institution décrit un manuscrit numérisé dans un fichier JSON, le **manifeste**. On le lit exactement comme corpus.json.

```
chemin = Path("donnees") / "Berne_318.json"
with open(chemin, "r", encoding="utf-8") as fichier:
    manifeste = json.load(fichier)
```

Un extrait simplifié (Berne, Burgerbibliothek, Cod. 318)

```
{ "@type": "sc:Manifest",
  "label": "Bern, Burgerbibliothek, Cod. 318",
  "metadata": [
    {"label": "Location", "value": "Bern"},
    ...
  ],
  "sequences": [
    { "canvases": [
        {"label": "Front cover", "width": 4872, "height": 6496},
        …  ]}]}
```

Les clés qui commencent par @ viennent du format JSON-LD : on y reviendra en séance 7.

Descendre niveau par niveau

- Un objet (dictionnaire) : on accède à une valeur par sa **clé**, ["label"].
- Une liste : on accède à un élément par son **indice**, [0].
- On enchaîne, de gauche à droite, **un niveau à la fois**.

```
manifeste["label"]
# 'Bern, Burgerbibliothek, Cod. 318'

for entree in manifeste["metadata"]:
    print(entree["label"], ":", entree["value"])
# Location : Bern
# Collection Name : Burgerbibliothek
# ...

canvases = manifeste["sequences"][0]["canvases"]
len(canvases)
# 281
canvases[0]["label"]
# 'Front cover'
canvases[4]["label"]
# '1r'
```

Notions clés à retenir

Les décorateurs

- Une fonction est une valeur : siecle désigne la fonction, siecle(1210) l'appelle.
- Un **décorateur** reçoit une fonction, l'enveloppe dans une nouvelle fonction, et renvoie celle-ci.
- @annoncer est une abréviation de afficher_titre = annoncer(afficher_titre).
- *args et **kwargs permettent de décorer n'importe quelle fonction, quels que soient ses paramètres.

Le format JSON

- Un format **texte** pour enregistrer et échanger des données **imbriquées** : objets { } et listes [ ].
- **Sérialiser** : de Python vers JSON ; **désérialiser** : de JSON vers Python.
- Attention aux différences : True devient true, None devient null, le tuple revient en liste, et les sets comme les objets ne sont pas sérialisables.

Lire et enregistrer des fichiers

- with open(chemin, mode, encoding="utf-8") ferme le fichier automatiquement. Le mode "w" **écrase** le contenu.
- json.dump() et json.load() travaillent avec un fichier ; json.dumps() et json.loads() avec une chaîne.
- ensure_ascii=False conserve les accents, indent=4 rend le fichier lisible.
- pathlib construit des chemins valables sur tous les systèmes, et mkdir(exist_ok=True) crée un dossier sans erreur.

Parcourir un manifeste

- Un manifeste IIIF est un fichier JSON : on le lit avec json.load().
- On y descend **un niveau à la fois** : une clé pour un objet, un indice pour une liste.